1. Introducción

El presente documento describe la arquitectura, estructura interna y funcionamiento técnico del sistema desarrollado en lenguaje C++ para el análisis del rendimiento académico de estudiantes universitarios.

El sistema permite la lectura y procesamiento de archivos estructurados .lfp, la aplicación de cálculos estadísticos avanzados y la generación de reportes analíticos en formato HTML.

2. Requerimientos Técnicos

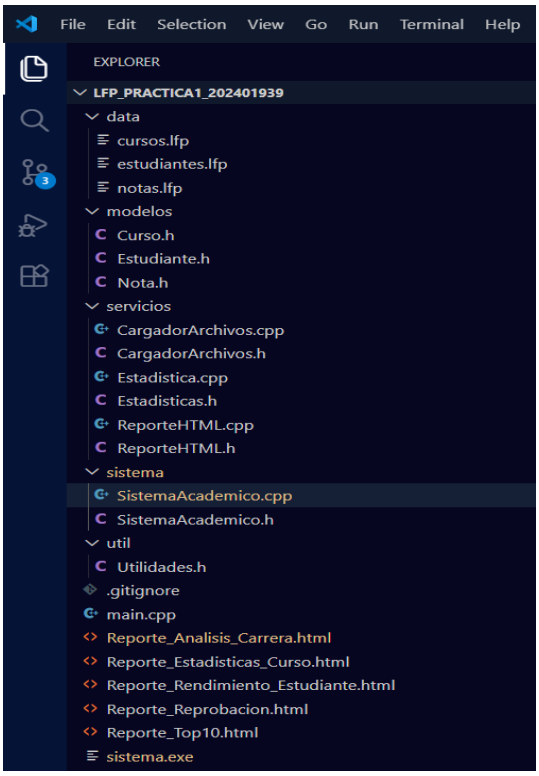
2.1 Lenguaje de Programación

- C++
- Uso de bibliotecas estándar (STL)

2.2 Herramientas Utilizadas

- Visual Studio Code
- Compilador g++
- Git y GitHub para control de versiones

2.3 Estructura del Proyecto



El diseño modular permite separación de responsabilidades y facilita mantenimiento y escalabilidad.

3. Arquitectura del Sistema

El sistema sigue una arquitectura modular dividida en:

- Capa de Modelos → Representación de datos
- Capa de Servicios → Lógica de procesamiento
- Capa del Sistema → Coordinación general
- Capa de Presentación → Generación de HTML
- Interfaz → Menú en consola

4. Descripción de Clases

4.1 Estudiante

Representa la información básica del estudiante:

- carnet
- nombre
- apellido
- carrera
- semestre

4.2 Curso

Representa los cursos del pensum:

- código
- nombre
- créditos
- semestre
- carrera

4.3 Nota

Representa la calificación obtenida por un estudiante en un curso:

- carnet
- códigoCurso

- nota
- ciclo
- año

4.4 Utilidades

Contiene métodos auxiliares como:

- split() → separa cadenas por delimitador

4.5 CargadorArchivos

Responsable de:

- Leer archivos .lfp
- Separar datos por comas
- Convertir texto a tipos numéricos
- Crear objetos
- Retornar vectores

4.6 Estadisticas

Contiene métodos para cálculos estadísticos:

- promedio()
- mediana()
- desviacionEstandar()
- percentil()

Implementa interpolación lineal para percentiles.

4.7 ReporteHTML

Encargado de:

- Generar archivos HTML
- Incluir estructura básica
- Insertar tablas dinámicas
- Aplicar estilos CSS simples

4.8 SistemaAcademico

Clase principal que:

- Almacena vectores de estudiantes, cursos y notas
- Relaciona información entre archivos
- Ejecuta cálculos
- Genera reportes

5. Lógica de Lectura de Archivos (Pseudocódigo)

Abrir archivo

Si no se puede abrir → mostrar error

Mientras haya líneas:

  Leer línea

  Separar por comas usando split

  Validar tamaño de datos

  Convertir datos a tipos correspondientes

  Crear objeto

  Agregar a vector

Cerrar archivo

Retornar vector

6. Lógica de Procesamiento de Reportes

Ejemplo: Estadísticas por Curso

Para cada curso:

  Crear vector vacío de notasCurso

```
    Para cada nota:
        Si codigoCurso coincide:
            Agregar nota al vector

    Si vector no está vacío:
        Calcular:
            promedio
            mediana
            desviacion
            percentil 25
            percentil 75
            mínimo
            máximo

        Agregar fila a HTML
```

7. Algoritmo de Percentiles

```
Ordenar datos
Calcular posición = (p/100)*(n-1)

Si posición es entera:
    Retornar valor directo

Si no:
    Interpolar entre valor inferior y superior
```

8. Relación entre Archivos

El sistema implementa relaciones manuales mediante búsquedas:

- Nota.carnet → Estudiante.carnet
- Nota.codigoCurso → Curso.codigo
- Curso.carrera → Estudiante.carrera

Esto permite análisis cruzado sin base de datos.

9. Observaciones Técnicas

- Se utilizó programación estructurada con modularidad.
- Se empleó STL para optimización.
- Se implementaron algoritmos de ordenamiento para ranking.
- El sistema genera reportes HTML independientes.
- El diseño permite escalabilidad futura (base de datos, interfaz gráfica).

10. Conclusión Técnica

El sistema cumple con:

- Lectura y procesamiento de archivos estructurados
- Aplicación de estadísticas descriptivas avanzadas
- Relación entre múltiples fuentes de datos
- Generación automatizada de reportes HTML

El diseño modular facilita mantenimiento, ampliación y reutilización del código.